

Trabajo Final: Sistema de Subastas Online con Cierre Temporal y Puja Segura

Descripción general

Diseñar e implementar el backend de una plataforma de subastas online donde los usuarios puedan publicar productos, realizar ofertas y competir hasta el cierre de cada subasta.

El sistema debe garantizar la integridad de las pujas, respetar reglas temporales, controlar concurrencia entre usuarios y proteger la información sensible.

Roles

El sistema debe contemplar los siguientes roles:

- USER: puede registrarse, consultar subastas y realizar pujas.
- SELLER: puede publicar productos y crear subastas.
- ADMIN: puede moderar publicaciones, suspender usuarios, cancelar subastas y resolver disputas.

Un usuario puede tener más de un rol.

Entidades principales

El modelo debe incluir, como mínimo:

- Usuario
 - Rol
 - Producto
 - Categoría
 - Subasta
 - Puja
 - Historial de estados
 - Notificación
 - Reclamo o disputa
-

Estados de una subasta

Una subasta puede encontrarse en los siguientes estados:

- BORRADOR
- PUBLICADA
- ACTIVA
- FINALIZADA
- CANCELADA
- ADJUDICADA
- EN_DISPUTA

No se permiten cambios directos de estado sin respetar las reglas de negocio.

Transiciones principales esperadas:

- BORRADOR → PUBLICADA
- PUBLICADA → ACTIVA
- ACTIVA → FINALIZADA
- ACTIVA → ADJUDICADA
- PUBLICADA / ACTIVA → CANCELADA
- ADJUDICADA → EN_DISPUTA
- EN_DISPUTA → ADJUDICADA / FINALIZADA / CANCELADA

Todo cambio de estado debe quedar registrado en un historial con fecha, usuario responsable y motivo cuando corresponda.

Reglas de negocio

Publicación de subastas

Un usuario con rol SELLER puede crear una subasta en estado BORRADOR.

Para ello debe indicar:

- Producto asociado
- Precio base
- Fecha y hora de inicio
- Fecha y hora de cierre
- Incremento mínimo de puja
- Categoría
- Descripción

El sistema debe validar que la fecha de cierre sea posterior a la fecha de inicio y que las fechas se almacenen en UTC.

Inicio y cierre automático

Una subasta pasa de PUBLICADA a ACTIVA cuando se alcanza su fecha de inicio.

Al alcanzar la fecha de cierre:

- Si no tiene pujas válidas, pasa a FINALIZADA.
- Si tiene pujas válidas, pasa a ADJUDICADA.

Estos cambios deben resolverse desde el backend, sin depender del cliente. Pueden implementarse mediante tareas programadas, validaciones al consultar la subasta o mecanismos equivalentes.

Registro de pujas

Un usuario puede ofertar solo si:

- La subasta está ACTIVA.
- La fecha actual del servidor es menor que la fecha de cierre.
- El usuario no es el vendedor.
- El usuario no está bloqueado.
- La subasta no está cancelada, finalizada, adjudicada ni en disputa.

Si es la primera puja, el monto debe ser mayor o igual al precio base.

Si ya existen pujas, el monto debe ser mayor o igual a:

`monto actual + incremento mínimo`

Los importes deben almacenarse usando tipos decimales exactos, no tipos de punto flotante.

Concurrencia de pujas

El sistema debe contemplar que varios usuarios pueden ofertar al mismo tiempo sobre una misma subasta.

Al registrar una puja, el backend debe validar el monto contra la última puja confirmada en la base de datos, no contra el valor que el cliente tenía visible.

La operación de registrar una puja debe ser transaccional y debe garantizar:

- Verificar que la subasta esté activa.
- Verificar que no haya finalizado.
- Obtener la última puja confirmada.
- Validar el monto mínimo requerido.

- Registrar la nueva puja.
- Actualizar el monto actual.
- Actualizar el usuario que va ganando.

Para resolver la concurrencia se podrá utilizar bloqueo pesimista, bloqueo optimista o mecanismos equivalentes provistos por la base de datos u ORM.

Cancelaciones

Una subasta solo puede ser cancelada por el vendedor si todavía no recibió pujas.

Una subasta con pujas solo puede ser cancelada por un administrador.

Toda cancelación debe registrar motivo, usuario responsable y fecha.

Privacidad

Mientras la subasta está activa:

- Los usuarios pueden ver el monto actual, ósea el valor más alto ofertado hasta ese momento, o el precio base si todavía no hubo pujas.
- Cada usuario puede consultar sus propias pujas.
- No debe exponerse la identidad completa de todos los oferentes.
- El administrador puede ver el historial completo.

Una vez finalizada o adjudicada, el vendedor puede consultar el historial completo de pujas.

Disputas

El vendedor o el ganador pueden abrir una disputa sobre una subasta adjudicada.

Al abrirse una disputa, la subasta pasa a EN_DISPUTA.

El administrador puede resolver la disputa y modificar el estado final de la subasta.

La disputa debe registrar motivo, descripción, usuario que la inicia, fecha de creación y resolución administrativa.

Requerimientos funcionales

El sistema debe permitir:

- Registro y login de usuarios con JWT.

- Hash seguro de contraseñas.
- Gestión de roles USER, SELLER y ADMIN.
- Bloqueo de usuarios por parte del administrador.
- Alta, edición y eliminación de productos.
- Creación, publicación, cancelación, listado y consulta de subastas.
- Registro de pujas.
- Consulta de pujas propias.
- Consulta del historial de pujas según permisos.
- Determinación del ganador.
- Registro del precio final y fecha de adjudicación.
- Generación de notificaciones al ganador y al vendedor.
- Apertura y resolución de disputas.

Las notificaciones pueden implementarse como registros en base de datos. No es obligatorio enviar emails, SMS ni notificaciones push.

Requerimientos no funcionales

El sistema debe cumplir con:

- Validaciones del lado servidor.
- Almacenamiento de fechas en UTC.
- Uso de transacciones para registrar pujas.
- Control de concurrencia en pujas.
- Auditoría de cambios de estado.
- Control de permisos por rol.
- No confiar en horarios enviados por el cliente.
- No confiar en montos actuales enviados por el cliente.
- Manejo adecuado de errores.
- Respuestas claras ante operaciones inválidas.

En ambiente productivo se recomienda HTTPS. Para ambiente académico o local se acepta HTTP.

Criterios de evaluación

Se evaluará especialmente:

- Correcto modelado de entidades y relaciones.
- Uso adecuado de autenticación y autorización.
- Correcto manejo de estados.

- Validación de reglas temporales.
- Registro transaccional de pujas.
- Resolución de concurrencia.
- Auditoría de cambios importantes.
- Claridad de endpoints.
- Manejo correcto de errores.
- Separación adecuada de responsabilidades en el backend.

Alcance del Proyecto

Para la aprobación del presente Trabajo Práctico Integrador, **es obligatorio y como requisito mínimo el diseño y desarrollo completo del Backend (API REST, Base de Datos, Seguridad y Lógica de Negocio)** que dé soporte a todos los requerimientos especificados en este documento.

Nota sobre el Frontend: El desarrollo de una interfaz gráfica (Frontend) **no es obligatorio**, pero queda a libre elección de cada equipo utilizar el lenguaje o framework que deseen. La implementación de pantallas funcionales para interactuar con la API **sumará puntos considerables en la rúbrica final de evaluación.**

Etapas y Cronograma de Trabajo

El desarrollo del proyecto se dividirá de forma estricta en las siguientes etapas consecutivas:

- **Etapas y Cronograma de Trabajo**

Etapas y Cronograma de Trabajo

Antes de iniciar cualquier actividad de desarrollo, los estudiantes deberán registrar el equipo en el campus virtual. El TPI se realizará en **grupos de 2 a 4 integrantes.**

- **Etapas y Cronograma de Trabajo**

- **Etapas y Cronograma de Trabajo**

Se coordinará una instancia presencial con el **Profesor (actuando en el rol de Cliente)**. En esta sesión, los equipos podrán:

- Evacuar dudas sobre el funcionamiento de las reglas de negocio o el alcance de los requerimientos.
- Negociar y redefinir el alcance en caso de que detecten que algún requisito técnico o funcional es inviable para los tiempos estipulados.

- **Etapla 4: Período de Desarrollo**

Finalizada la instancia de negociación, los equipos quedarán formalmente habilitados para comenzar la codificación. A partir de ese momento, dispondrán hasta el **30 de junio** para la finalización del trabajo.

En esta etapa deberán en primera instancia formar el modelado de datos para dar soporte a todos los requerimientos anteriormente definidos. Además deberán presentar de forma semanal avances correspondientes al desarrollo.

Entregables y Evaluación Final

Llegada la fecha límite, cada grupo deberá realizar la entrega formal y la posterior defensa del proyecto:

1. **Código Fuente y Documentación:** Se deberá entregar a través del campus el **enlace al repositorio de código (ej. GitHub/GitLab)** que contenga el proyecto, acompañado de la documentación técnica y funcional que el equipo considere necesaria para respaldar su arquitectura y diseño (diagramas de base de datos, colección de Postman, etc.).
2. **Exposición Funcional (Defensa):** El equipo realizará una presentación en vivo del sistema ante el docente. En esta instancia deberán:
 - Demostrar el funcionamiento del software mediante un **flujo de uso significativo** (por ejemplo: registro de un usuario, creación de un producto, carga de una subasta, registro de pujas, adjudicación, etc).
 - Explicar a grandes rasgos la experiencia de desarrollo, detallando los principales inconvenientes técnicos o de lógica que surgieron y cómo lograron resolverlos.

Rúbrica de Evaluación

1. Instancia de Consulta y Negociación (Reunión con el Cliente)

Este módulo evalúa la preparación previa, la madurez profesional y la capacidad de análisis del equipo antes de escribir la primera línea de código.

Criterio	Excelente	Bueno	Regular	Insuficiente
Comprensión del Negocio y Preparación	Demuestran una lectura exhaustiva del documento. Identifican inconsistencia	Leyeron la consigna y comprenden el flujo general del sistema, pero las dudas	La lectura fue parcial. El equipo asiste a la reunión a que el profesor les explique la consigna desde	No se presentan a la instancia de consulta o demuestran un desconocimien

	s, desafíos técnicos o zonas grises de la lógica temporal o de estados.	presentadas son superficiales o se resuelven con una relectura simple del texto.	cero, demostrando falta de análisis previo.	to total de los requerimientos de la plataforma.
Capacidad de Negociación Técnica	Proponen alternativas válidas, justificadas y viables frente a requisitos complejos (ej. manejo de zonas horarias o transiciones automáticas), defendiendo su postura con criterio.	Expresan qué requerimientos les parecen difíciles o inviables, pero no logran proponer alternativas de solución claras o viables para el negocio.	Se limitan a pedir que se eliminen requerimientos por falta de tiempo o complejidad, sin argumentación técnica ni propuestas de diseño alternativas.	No muestran capacidad de análisis crítico ni proponen adaptaciones sobre el alcance del sistema

2. Desarrollo del Producto (Evaluación del Repositorio y Código)

Este módulo evalúa la calidad técnica de la entrega final del backend (no aplica para el frontend) tras las 3 semanas de desarrollo.

Criterio	Excelente	Bueno	Regular	Insuficiente
Cumplimiento Funcional y Reglas de Negocio	Cumple con el 100% de los requerimientos funcionales del backend. Las reglas críticas del dominio funcionan correctamente : publicación	Cumple con la mayoría de los requerimientos funcionales. Existen fallas menores en casos borde, por ejemplo validaciones incompletas de fechas, errores	El backend es inestable o incompleto. Faltan módulos funcionales importantes, por ejemplo gestión de disputas,	El sistema no compila, no ejecuta o no cumple con los requerimientos mínimos indispensables para simular una

	de subastas, cambio de estados, validación temporal, registro de pujas, adjudicación, cancelaciones , privacidad de oferentes y resolución de disputas.	menores en cambios de estado o restricciones de edición parcialmente aplicadas.	adjudicación, historial de estados, cancelaciones o privacidad de pujas. Varias reglas de negocio críticas fallan o se aplican de forma inconsistente.	subasta online. No permite registrar usuarios, crear subastas o realizar pujas válidas.
Arquitectura de Software y Persistencia	Código limpio, modularizado y ordenado. Correcta aplicación de la arquitectura lógica, manejo de excepciones global, tipado estricto y consistencia en el almacenamiento o (UTC).	Estructura funcional, pero con acoplamiento innecesario. Lógica de negocio mezclada en controladores o controladores con excesiva responsabilidad. Base de datos íntegra.	Arquitectura confusa o desorganizada. Falta de consistencia en el código, validaciones inexistentes en el servidor o problemas severos de integridad referencial.	Código espagueti sin ningún patrón de diseño aparente. No se respetan las buenas prácticas mínimas de desarrollo de APIs REST.
Seguridad (Auth) e Integridad	Implementación robusta de JWT. Restricción estricta de endpoints según rol (ADMIN/SELLER/USER/) y contraseñas hashadas de	Sistema de autenticación funcional, pero con vulnerabilidades menores (ej. algunos endpoints de ADMIN son accesibles por USER, o las contraseñas	El login funciona de manera precaria. No hay una diferenciación clara y segura de los roles en los endpoints críticos del sistema.	Las contraseñas se almacenan en texto plano, no hay seguridad implementada o el sistema de autenticación

	manera segura en la BD.	usan algoritmos obsoletos).		n no funciona.
Valor Agregado: Frontend e Interfaz (Plus)	(Suma puntos extra) Presenta una interfaz funcional e intuitiva que consume la API y permite operar el flujo principal: login, productos, subastas, pujas y resultados..	(Suma puntos extra) Presenta pantallas básicas conectadas a la API para probar parte del flujo, como login, listado/detalle de subastas y registro de pujas.	No presenta frontend o las pantallas realizadas son incompletas, aisladas o no se conectan correctamente con la API. No afecta el mínimo obligatorio del backend.	No aplica penalización , ya que el frontend es opcional y solo suma como valor agregado.

3. Presentación del Sistema y Defensa

Este módulo evalúa las habilidades blandas del equipo, la comunicación técnica y su honestidad intelectual sobre el proceso de desarrollo.

Criterio	Excelente	Bueno	Regular	Insuficiente
Demostración Funcional en Vivo	Exponen un flujo de uso continuo, fluido y representativo . Dominan la herramienta.	Muestran el flujo completo del sistema, pero la demo se interrumpe por errores menores de configuración en vivo o datos mal precargados que logran sortear.	La demostración es caótica. Prueban endpoints sueltos (ej. desde Postman) pero no logran hilvanar un flujo de negocio completo o representativo .	El sistema falla críticamente durante la presentación (crash) impidiendo evaluar el flujo funcional del software.

Análisis de la Experiencia e Inconvenientes	Explican con solidez técnica los problemas sufridos (bloqueos lógicos, concurrencia, manejo del tiempo), fundamentando con claridad la solución que adoptaron para resolverlos.	Mencionan los inconvenientes surgidos durante las 3 semanas de desarrollo, pero la explicación de cómo los solucionaron carece de profundidad técnica.	Se limitan a decir que "tuvieron problemas" o que "no les salía", demostrando falta de reflexión sobre el proceso de aprendizaje y resolución.	No son capaces de identificar fallas, justificar decisiones de diseño o evaden hablar sobre las dificultades del proceso.
Dinámica de Grupo y Respuestas Técnicas	Todos los integrantes demuestran un conocimiento pleno de la totalidad del código y responden preguntas conceptuales o técnicas individuales con absoluta solvencia.	La participación está algo centralizada en un programador principal, aunque el resto del grupo responde aceptablemente sobre las partes asignadas.	Distribución del conocimiento muy deficiente. Hay integrantes del grupo que no pueden responder preguntas básicas sobre el funcionamiento del código entregado.	Evidente falta de trabajo en equipo. Algún integrante demuestra un desconocimiento total del proyecto, sugiriendo que no participó en el desarrollo.